

RugRadar

A crypto scam-token risk scorer

Paste any crypto address. Get a clear risk score,
a risk level, and plain-English reasons why.

What it covers	How
7 blockchains	Ethereum, BSC, Polygon, Base, Arbitrum, Bitcoin, Litecoin
12 data sources	Security APIs, blockchain nodes, and community blocklists
3 services	React frontend, Express backend, Python scoring engine
~2,800 lines	14 Python modules, 12 test scripts, 88 commits

[Project documentation](#) — architecture, tech stack, working flow, usage, and roadmap

1. What This Project Does

Every day people lose money to fake crypto tokens. A scammer makes a token, people buy it, and then they find out they cannot sell it. Or the scammer removes all the money from the trading pool and disappears. These are called **honeypots** and **rug pulls**.

The hard part for a normal person is that checking a token is very technical. You would need to read smart contract code, look at who holds the tokens, and check if the trading pool is locked. Most people cannot do this.

RugRadar does that checking for you. You paste an address. It gives you back three things:

- A **score from 0 to 100** — higher means more dangerous.
- A **risk level** — Low, Medium, High, or Unknown.
- A **list of reasons** in normal English, so you know exactly why.

Real example from the live app

Address: 0x45dAc6C8776E5Eb1548d3CdcF0C5f6959e410c3A

Score: 100 / 100 Level: HIGH RISK

Why:

- This token is flagged as a honeypot: you may be able to buy it but unable to sell it.
- The contract owner can directly change any wallet's balance.
- The contract can mint new tokens at any time.
- The top 10 wallets hold 100% of the total supply.

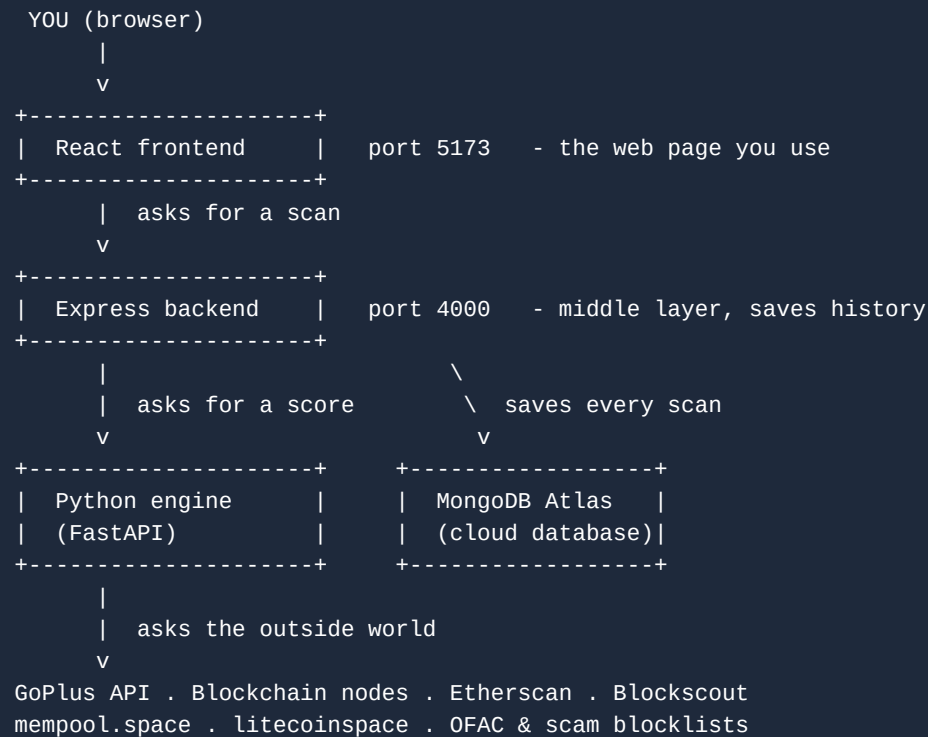
The one rule that shapes the whole project

A safety tool that says "**safe**" when it really means "**I could not check**" is worse than no tool at all. People would trust it and lose money.

So RugRadar has a fourth risk level: **Unknown**. If no data source knows anything about an address, we say so clearly. We never show a green badge just because we found nothing.

2. System Architecture

The project is split into three separate programs. Each one has a single job. They talk to each other over HTTP, so any one of them can be changed or replaced without breaking the others.



Why three separate services?

Service	Job	Why it is separate
React (client)	Shows the page, takes the address you type, displays the result.	Front-end work changes often. Keeping it separate means design changes never touch the scoring logic.
Express (server)	Passes requests to the Python engine and saves every scan to the database.	Gives one clean address for the frontend to talk to, and is the natural place to add login or rate limits later.
FastAPI (ml-service)	Does all the real work: gathers data from 12 sources and calculates the score.	Python is best for data work and machine learning. Keeping it alone means the future ML model drops straight in.

Design choice: fail softly, never crash

Extra information is always optional. If a blockchain node is slow, or a blocklist will not download, or the history service is busy, the scan still finishes and still gives a score. You just see less detail, plus a short note explaining what was missing.

The only thing that stops a scan is when we genuinely cannot judge the address at all — and then we say that out loud instead of guessing.

3. Tech Stack

We deliberately used very few libraries. Modern Node.js and Python already include most of what a project like this needs, and every extra library is one more thing that can break or need updating.

Frontend

Tool	Version	What it is for
React	19	Builds the user interface
Vite	8	Runs the dev server and builds the final files. Very fast.
Tailwind CSS	4	Styling, written directly in the HTML

Backend

Tool	Version	What it is for
Node.js	24	Runs the backend
Express	4	Handles web requests
Mongoose	8	Talks to the MongoDB database
MongoDB Atlas	cloud	Stores the history of every scan

Scoring engine

Tool	Version	What it is for
Python	3.14	Runs the scoring engine
FastAPI	latest	Builds the API, and creates docs automatically
uvicorn	latest	The web server that runs FastAPI
requests	latest	Calls the outside APIs

Libraries we chose NOT to install

axios — Node 24 already has `fetch` built in.

nodemon — Node 24 already has `--watch` built in.

pytest — our tests are plain Python scripts that print readable results.

Result: the backend has only **4** dependencies and the engine only **4**. The whole project installs in seconds.

4. Data Sources — Where The Answers Come From

This is the most important part of the project. No single service knows everything about crypto scams, so RugRadar asks **twelve different sources** and combines what they say.

Group 1 — Security analysis (GoPlus)

GoPlus runs blockchain nodes and actually simulates a buy and a sell on each token to find honeypots. This is the expensive, difficult work, and they give the results away for free.

API	What it tells us	Key needed?
token_security	Honeypot, mintable, hidden owner, buy/sell tax, liquidity locks, top holders — about 34 facts per token	No
address_security	If an address is linked to sanctions, phishing, money laundering, mixers, theft — 19 flags	No
supported_chains	Which blockchains each API actually covers	No

Group 2 — Live blockchain facts (public nodes)

Source	What it tells us	Key needed?
publicnode.com RPC (5 chains)	Current balance, whether the address is a contract or a wallet, size of the contract code	No

Group 3 — Transaction history (needs an indexer)

A blockchain node can tell you what is true *right now*, but it cannot list every past transaction. That needs a service that has indexed the whole chain.

Source	What it tells us	Key needed?
Etherscan V2	Past transactions, token transfers, contract creation date	Free key
Blockscout	The same, used as a backup when Etherscan cannot help	No

Group 4 — Bitcoin and Litecoin

Source	What it tells us	Key needed?
mempool.space	Bitcoin balance, total received and sent, transaction count	No
litecoinspace.org	The same for Litecoin	No

Group 5 — Community blocklists (3,847 known bad addresses)

Source	What it tells us	Entries
OFAC sanctions list	Addresses under official government sanctions — the strongest evidence there is	120 ETH 532 BTC 13 LTC
ScamSniffer	Known scam and phishing addresses	2,530
MyEtherWallet darklist	Reported scams, with a written description of each one	652

These are plain files on GitHub. We download them once when the server starts and keep them in memory, so checking an address against all 3,847 entries is instant.

Why GoPlus needs no API key at all

An API key normally answers one of three questions: *who are you?*, *what are you allowed to see?*, or *who do I charge?*

None of those apply here. Everything GoPlus reports comes from **public blockchain data**. Anyone could work it out by running their own blockchain node. GoPlus simply did the expensive part and shares the answers. To stop abuse they limit how many requests one internet connection can make, instead of asking who you are.

Practical result: your teammate can clone this project and run it with zero signups.

5. How A Scan Works, Step By Step

Step 1 — Work out what kind of address it is

Before anything else, we look at the shape of the text you pasted:

Looks like	Chain family
0x followed by 40 hex characters	EVM (Ethereum, BSC, Polygon, Base, Arbitrum)
Starts with 1, 3 or bc1	Bitcoin
Starts with L, M or lt c1	Litecoin
Anything else	Rejected with a clear message listing the accepted formats

Step 2 — Find which blockchain it is actually on

You do not have to choose the chain. We search five blockchains **at the same time** and then **rank** the results by how much real information came back.

Why ranking matters — a real example

The same address can exist on several blockchains as **completely different things**.

Take 0x0E09FaBB...CE82:

- On **BSC** it is PancakeSwap Token, with 1.9 million holders.
- On **Ethereum** it is an empty contract — no name, no symbol, no holders.

Both chains reply. If we simply took the first answer, we would report the empty Ethereum contract and get everything wrong. So each result gets a quality score based on whether it has a name, a symbol, real holders and trading activity. The best one wins.

Step 3 — Choose how to score it

If the address is...	Then we...
A token	Score it with token rules (honeypot, taxes, liquidity, holder concentration) and check the contract against malicious-address records. Both scores add up.
A wallet or plain contract	Score it on malicious-address records and community blocklists only. There is no token to analyse.
A Bitcoin/Litecoin address	Score it on sanctions lists plus behaviour patterns. These chains have no smart contracts at all.

Step 4 — Add the extra detail

- **Live balance** and whether it is a contract or a wallet, read from a blockchain node.
- **Token information** — name, symbol, holder count, who created it, which exchanges it trades on, how much liquidity is locked, and the top 10 holders.

- **Transaction history** — recent transactions, token transfers, and when the contract was created.

An important efficiency note: almost all the token detail was **already inside the response we used for scoring**. We were throwing it away. Showing it costs zero extra API calls.

Step 5 — Save it

The Express backend stores every completed scan in MongoDB, so a history page can show what has been checked before.

6. How The Score Is Calculated

The scoring is a **transparent rule system**, not a black box. Every single point in the final score can be traced back to a specific, stated reason. This matters: a user who is told "73/100" with no explanation learns nothing.

Each rule that triggers adds points **and** one sentence of explanation. The total is capped at 100. All the numbers live in one dictionary at the top of the file, so they can be adjusted without touching any logic.

Token risk weights

Risk found	Points	Why it matters
Honeypot	65	You can buy but not sell. Your money is simply gone.
Hidden owner	15	Nobody can tell who controls the token.
Owner can change balances	15	The owner can take tokens straight out of your wallet.
Can mint new tokens	12	The supply can be inflated, making your tokens worthless.
Ownership can be taken back	12	"Ownership renounced" turns out to be a lie.
Source code not published	10	Nobody can check what the contract really does.
Transfers can be paused	10	The owner can freeze all trading whenever they want.
High sell tax (over 10%)	10	A large slice of your money is taken when you sell.
Liquidity not locked (under 50%)	10	The trading pool can be drained — a rug pull.
Top 10 wallets hold over 50%	8	A few people can crash the price at any moment.
Blacklist function exists	8	The owner can block your wallet from selling.
Creator holds over 20%	8	The creator can dump a huge amount on the market.

Risk levels

Score	Level	Meaning
0 – 29	Low	Checked, and nothing serious found
30 – 59	Medium	Some real warning signs
60 – 100	High	Serious danger — do not buy
—	Unknown	We could not check it. Not the same as safe.

Three decisions worth explaining

1. A honeypot is worth 65 points, not 40

We first gave a honeypot 40 points. Then we tested a real one called LILPEPE. Every other field looked perfectly clean, so it scored 40 and was labelled **Medium** risk.

That is wrong. If you cannot sell a token, nothing else about it matters. We raised the weight to 65, which is enough to reach **High** on its own. It now scores 65/High.

2. Missing data adds risk instead of being ignored

If a data source does not report a field, that field is not silently skipped. It adds a small **+2** and is named in the reasons.

A token that nobody can report on is genuinely less trustworthy than one that checks out cleanly. Ignoring gaps would quietly hide exactly the tokens we should worry about.

3. Behaviour patterns alone can never reach High

For Bitcoin and Litecoin we can only look at behaviour — things like a wallet that received a lot of coins and now holds nothing.

That pattern is common for scam payout wallets. But it is **also exactly what any normal wallet looks like** after someone spends their money. So behaviour signals are capped below High. Only hard evidence, like a sanctions listing, can produce a High verdict. This stops the app confidently accusing innocent people.

7. How To Run And Use It

Setup (once)

```
# 1. The Python scoring engine
cd ml-service
python -m venv venv
venv\Scripts\Activate.ps1
pip install -r requirements.txt

# 2. The Express backend
cd ../server
npm install
cp .env.example .env          # then add your MongoDB address

# 3. The React frontend
cd ../client
npm install
```

Running it (three terminals)

```
# Terminal 1 - scoring engine
cd ml-service; venv\Scripts\Activate.ps1; uvicorn app.main:app --reload --port 8000

# Terminal 2 - backend
cd server; npm run dev

# Terminal 3 - frontend
cd client; npm run dev

Then open: http://localhost:5173
```

Order matters: the frontend calls the backend, which calls the engine. If the engine is not running you will see a clear message saying so, not a blank screen.

Using it

- Paste any crypto address into the box.
- Leave the dropdown on **Auto-detect chain** — it works out the blockchain itself.
- Press **Scan address**.
- Read the score, the level, and the list of reasons.

Test addresses (all verified against the live app)

High risk

Address	Result
0x0330070FD38Ec3bB94F58FA55D40368271E9e54A	100 / High — on the OFAC sanctions list

Address	Result
0x45dAc6C8776E5Eb1548d3CdcF0C5f6959e410c3A	100 / High — honeypot
0x2c27cf135980cd8f05008f1443e1e4a14d33329f	100 / High — token literally named "Honeypot"
0xA37Bb2AAac324EB974ea46729217040A23Eae996	65 / High — clean-looking honeypot
123WBUDmSJv4GctdVEz6Qq6z8nXSKrJ4KX	100 / High — sanctioned Bitcoin address
ltc1qr8ntsedq8tv0svmxqhzvdccl5k7kntdmnhwep7	100 / High — sanctioned Litecoin address

Low risk / normal

Address	Result
0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48	19 / Low — USDC
0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2	10 / Low — WETH
0x0E09FaBB73Bd3Ade0a17ECC321fD13a19e81cE82	26 / Low — auto-detects to BSC
0x95aD61b0a150d79219dCF64E1E6Cc01f0B64C4cE	SHIB — 26 exchange pools, full history

Special cases

Address	Result
0xc198c3b7b970cbac315614cf27a1b7eb332048d6	Unknown — a real contract on BSC that no source can analyse
0xd8dA6BF26964aF9D7eEd9e03E53415D37aA96045	Smart account (EIP-7702) — shows what the wallet delegates to
hello-world	Clear error naming all three accepted address formats

8. What Makes RugRadar Different

Most scam checkers are a thin wrapper around a single API. If that one service does not know the answer, the tool shows a green tick and the user gets hurt. RugRadar is built around the opposite idea.

1. It admits when it does not know

This is the biggest difference, and it came directly from a real test. A user checked a BSC address that other security firms had blacklisted. Our app said "**Low risk**" with a green badge, because the source we asked had no record of it.

The address scored zero only because **nobody had checked it** — but the screen made that look like a clean bill of health. We added a fourth level, **Unknown**, shown in grey with this wording:

```
"This is a deployed contract, but no security provider has a record of it, so it could not be analysed. That is NOT the same as being safe - unverified contracts are exactly where new scams live."
```

No competitor we looked at makes this distinction.

2. Twelve sources, with automatic fallbacks

Every source has gaps. Etherscan's free plan does not cover BSC or Base, so we fall back to Blockscout, which covers Base without any key. When **no** source covers a chain, we print the reason instead of showing an empty list that would suggest the address has no history.

3. It finds the right blockchain by itself

The user never has to know which chain a token is on. We check five chains at once and rank the answers by data quality, which correctly picks BSC over Ethereum for PancakeSwap instead of reporting an empty contract.

4. It covers Bitcoin and Litecoin, not just Ethereum

These chains have no smart contracts, so token checks are meaningless there. Rather than pretend otherwise, RugRadar switches to sanctions lists plus an activity profile, and says on screen why the token checks do not apply.

5. It detects modern smart-account wallets

Since the 2025 Ethereum upgrade (EIP-7702), an ordinary wallet can carry code. A naive check reports every such wallet as a "contract". RugRadar recognises the 23-byte delegation marker and shows **what the wallet delegates to** — which is itself a security signal, because attackers trick users into delegating to wallet-draining contracts.

6. Every point is explainable

There is no hidden model. Each rule states its own reason in plain English, so a user learns what is actually wrong rather than just seeing a number.

7. It runs with zero API keys

The entire app works without a single signup. An optional free Etherscan key only improves the transaction history display. Judges can clone it and run it immediately.

9. Real Bugs We Found By Testing Against Live Data

We tested every step against real blockchain data rather than mock data. That caught nine genuine bugs — several of which would have made the app dangerously wrong. Each one now has a test so it cannot come back.

What went wrong	Why it was dangerous	The fix
A confirmed honeypot scored 40 and was labelled Medium	The most dangerous kind of token was not shown as high risk	Raised the honeypot weight to 65
A scam contract reported holders as 1.15e+52%	A broken-looking number destroys user trust instantly	All percentages are clamped to 0–100
When the API was rate-limited, we reported "not a token"	A real scam token would score 0 and look completely safe	Rate limits now raise an error instead of a false all-clear
Addresses with no token record defaulted to Ethereum	A live BSC scam contract was reported as an empty wallet	We now find the chain where the address really has code
A score of 56 was displayed as Low	The number and the label contradicted each other	The level is recalculated after scores are combined
The backend hardcoded the chain to 1	Auto-detection was silently switched off for every web request	Backend now defaults to auto, matching the engine
An unverified contract showed a green Low risk badge	"We found nothing" was displayed as "this is safe"	Added the Unknown level with a warning
A modern smart-account wallet was called a contract	Every EIP-7702 wallet would be described wrongly	Detect the delegation marker and show the delegate
A mistyped Bitcoin address returned 502 server error	A user typo looked like our service had crashed	Now a clear 400 explaining the checksum failed

Why this list belongs in the documentation

Every one of these bugs was invisible in normal testing. They only appeared because we checked real tokens, real scam addresses and real wallets against live APIs.

Four of the nine would have caused the app to call something **safe when it was not**. For a security tool, that is the only failure that truly matters — and it is the failure mode the whole design now guards against.

10. Honest Limitations

We would rather state these clearly than have a judge discover them in a demo.

Limitation	Explanation
Not every scam is known	Our sources do not know every bad address. A brand new scam may show as Unknown. That is why Unknown exists and is never shown as green.
Free tier rate limits	GoPlus limits how many requests one connection can make. Auto-detect uses about 6 calls per scan, so rapid repeated scans can be temporarily blocked.
No BSC transaction history	Etherscan's free plan excludes BSC and there is no public Blockscout for it. The risk score still works; only the history panel is missing, and it says why.
Token holdings not available	Listing everything a wallet holds is a paid-only feature on Etherscan.
Bitcoin/Litecoin signals are weak	With no smart contracts, only sanctions lists and behaviour are available. This is why behaviour alone can never produce a High verdict.
Solana and Tron not supported	Their addresses use a different format. Our validation is currently EVM and Bitcoin-style only.

11. What We Would Build Next

Ordered by how much each would improve the product, with the effort involved.

Priority 1 — A machine learning model

Item	Detail
What	Train an XGBoost or Random Forest model on a labelled dataset of real scam and safe tokens, using the same features we already collect.
Why it matters	The rule weights are currently chosen by human judgement. A trained model would learn which combinations of signals actually predict scams, and could catch new scam patterns the rules miss.
The honest part	We would not train on the honeypot flag, because that would just teach the model to copy an answer it already has. We would use independent scam labels.
Safety design	The rule-based scorer stays as a fallback, so a model failure never takes the product down. We would report accuracy, precision, recall and feature importance.
Effort	Medium. The feature pipeline already exists — the missing piece is a labelled dataset.

Priority 2 — Browser extension: warn before you buy

The biggest weakness of any website checker is that the user has to remember to visit it. An extension would watch for token addresses on exchange pages and show the risk score **automatically, at the moment it matters** — before the buy button is pressed. This turns the project from a tool people must remember into protection that is simply always on.

Priority 3 — Wallet warning before signing

Most money is lost at the moment of approving a transaction. Scanning the destination address at signing time, and warning on dangerous approvals, would stop losses at the exact point they happen. Our EIP-7702 delegation detection is already groundwork for this: malicious delegations are an active wallet-draining technique.

Priority 4 — Better coverage

- **HashDit integration** — a BSC-focused security service that knew about the exact address our current sources missed. This would directly close the one confirmed gap.
- **Solana and Tron** — already supported by GoPlus; we only need to handle their address formats.
- **More blocklists** — each additional community list adds coverage for free.

Priority 5 — Features that make the demo shine

Feature	Why a judge would care
Watch list and alerts	Re-check saved tokens on a schedule and alert if the risk rises. Scams often look fine at launch and turn bad later — a one-off check cannot catch that.
Confidence indicator	Show how many sources agreed. "5 of 5 sources checked" is far more trustworthy than a bare number.
Telegram / Discord bot	Crypto communities live in these apps. Checking a token where people actually discuss it removes all friction.
Batch and portfolio scan	Paste a whole wallet and check every token it holds at once.
Risk history graph	We already save every scan. Showing how a token's risk changed over time turns saved data into a genuine feature.
Public API	Let other developers build on our scoring. This makes the project infrastructure, not just an app.

The strongest thing to say in a pitch

Most scam detectors answer **"is this token bad?"**

RugRadar answers a harder and more useful question: **"how much do we actually know about this token, and how confident should you be?"**

That is why it has four risk levels instead of three, why it asks twelve sources instead of one, and why behavioural guesses can never reach a High verdict. Anyone can display a number. Knowing the limits of your own answer is the harder engineering problem — and it is the part that actually keeps users safe.

12. Summary

Question	Answer
What is it?	A web app that checks any crypto address and returns a 0–100 risk score, a risk level, and plain-English reasons.
What is the architecture?	Three services: React frontend, Express backend with MongoDB, and a Python FastAPI scoring engine.
Where do the answers come from?	Twelve sources: GoPlus security APIs, public blockchain nodes, Etherscan, Blockscout, mempool.space, litecoinspace, and three community blocklists holding 3,847 known bad addresses.
Which chains?	Ethereum, BSC, Polygon, Base, Arbitrum, Bitcoin and Litecoin — with the chain detected automatically.
Do you need API keys?	No. It runs fully without any signup. One optional free Etherscan key improves the transaction history display.
What makes it unique?	It admits when it does not know. The Unknown level means "we could not check" is never displayed as "this is safe".
What is next?	A machine learning model, a browser extension that warns before you buy, and wallet-level warnings at the moment of signing.

Project statistics

Measure	Value
Lines of code	about 2,800
Python modules	14
Test scripts	12, each testing against live data
Git commits	88, all with clear conventional messages
Dependencies	4 backend, 4 engine, 4 frontend
Blocklist entries checked per scan	3,847
Blockchains supported	7
Data sources combined	12